National University
Of Computer and Emerging Sciences

Project Phase #3

Presented to

Dr. Basharat Hussain

In partial fulfillment

of the requirements for the course of

AI Product Development (AI - 4013)

By:

Hafsa Imtiaz, 22i-0959

Mahum Hamid, 22i-1009

Areeba Riaz, 22i-1244

Jawairia Waseem, 22i-1274

MealMind: Reducing Daily Meal Decision Fatigue

Phase 3: MVP Development

1. Product Scope and Alignment with Phases 1 & 2

During Phase 1 and Phase 2, our research consistently pointed to one core issue: **meal decisions are mentally exhausting**. Users are not short of food apps or recipes—they are overwhelmed at the exact moment they need to decide what to eat.

Students, working professionals, and busy households described the same problem. They often have limited time, a fixed budget, and partial ingredients, yet existing apps expect advance planning, grocery trips, or detailed tracking. This disconnect between real-life behavior and available tools guided our MVP scope.

MealMind was therefore designed around a single question:

> **"What can I eat right now?"**

Instead of building a large planning system, we focused on *immediate decision support*. The MVP addresses this moment in two practical ways:

- **Cooking at home**, using only the ingredients the user already has
- **Ordering food nearby**, using natural language instead of filters

This scope directly reflects our earlier research decisions and avoids features that were not validated by user behavior.

2. What We Built (Core Functionality)

The MealMind MVP delivers a **complete, working user flow** from login to recommendation, without relying on mock data or placeholders.

Fridge-First AI Chef

For users cooking at home, the AI Chef allows them to enter:

- Available ingredients
- Time constraints
- A PKR budget

Based on this, the system generates a **clear, step-by-step recipe** that fits the user's real situation. The focus is on practicality rather than perfection—quick meals, realistic steps, and simple nutritional indicators.

This feature directly reduces decision fatigue by turning uncertainty into an actionable plan.

Smart Local Search (Cravings)

For users who prefer ordering food, we built a natural language search experience.

Instead of selecting multiple filters, users type what they actually think, such as:

- “cheap burgers nearby”
- “chicken under 1500”

The system interprets the request, finds relevant nearby restaurants, and displays them on an interactive map with ordering options. This approach feels more intuitive and better matches how users express cravings in real life.

Personalization and Continuity

To keep the experience coherent, we implemented a unified user profile. Dietary preferences, allergies, and spice tolerance are saved once and automatically applied across:

- Recipe generation
- Chat interactions
- Future sessions

Users can also save recipes and restaurant results. These are synced in real time and backed up locally to ensure reliability during demos.

3. What We Intentionally Did Not Build

Equally important to what we built is what we *chose not to build*.

We intentionally excluded **pantry tracking and receipt scanning**. While these features sound useful, research showed that users dislike maintaining inventories, and such tools add friction instead of reducing it.

We also avoided **weekly meal planning**. These tools require upfront effort and quickly break when schedules change. Our focus remained on immediate, low-effort decisions.

Urdu localization was identified as valuable but deferred due to complexity and time constraints. Similarly, **payments and premium features** were excluded to keep the MVP focused on validation rather than monetization.

These decisions helped prevent overengineering and kept the MVP aligned with its core goal.

4. System Overview (High-Level Architecture)

The system follows a clean and understandable client–server architecture.

- The **frontend** handles user interaction, routing, visual design, and map rendering.
- The **backend** manages all AI logic, natural language processing, and secure API communication.

This separation improves security, makes testing easier, and keeps the system flexible for future phases. External services are used only where necessary, keeping the architecture simple and realistic.

5. Key Technical Decisions and Trade-offs

One major decision was to use a **decoupled frontend and backend** instead of a single monolithic framework. While this required extra setup, it improved clarity, security, and maintainability.

We chose **Firestore** for user data to support fast development and real-time updates. Although it limits complex relational queries, it fits the MVP's needs well.

Using AI for **natural language interpretation** added some latency, but greatly improved usability. This was balanced with clear loading states so the experience remained smooth.

Finally, we implemented **fallback mechanisms** to ensure the demo works even if external services fail. Reliability was prioritized over architectural perfection.

6. Known Limitations and Risks

The MVP is functional but not perfect.

AI-generated recipes may occasionally suggest unrealistic cooking times. Location-based pricing data can also be inconsistent in local markets.

From a technical perspective, the system relies on multiple external APIs, which could become costly at scale. Caching and optimization are planned for future iterations.

Testing currently focuses on core flows, with more comprehensive end-to-end testing planned later.

7. Assumptions This MVP Is Testing

This MVP is designed to validate three key assumptions:

1. Users prefer deciding meals based on what they already have
2. Natural language input feels easier than traditional filter-based interfaces
3. Users are willing to enter preference data if it is remembered and reused

Phase 4 will evaluate these assumptions using usage data, feedback, and engagement metrics.

Conclusion

Phase 3 was not about building as many features as possible—it was about building the *right* ones. MealMind's MVP delivers a focused solution to a validated problem, with a clear user flow and sensible technical choices.

This foundation allows Phase 4 to move beyond speculation and into real product-market fit evaluation. The MVP proves not just that we can build, but that we understand *what* to build and *why*.